PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

2.º curso · CFGS Desarrollo de Aplicaciones Multiplataforma

Ejercicios: widgets

Ejercicios para analizar, preguntar a la IA y explicar

Cuaderno de ejercicios

Nombre: _____ Grupo: _____ Fecha: _____

Índice

1. Antes de empezar 3

Ejercicio 1. El detective de árboles 6

Ejercicio 2. ¿Con estado o sin estado? 8

Ejercicio 3. El contador que no cuenta 10

Ejercicio 4. Los datos bajan, los eventos suben 12

Ejercicio 5. Revisor de código de la IA 14

Antes de entregar 15

1. Antes de empezar

En estos cinco ejercicios vas a trabajar la parte básica de los widgets de Flutter: el árbol de widgets, los widgets sin estado y con estado, `setState()` y la comunicación entre widgets. Puedes y **debes** usar una herramienta de inteligencia artificial (IA), pero no para que haga el trabajo por ti, sino como un compañero al que preguntar, al que corregir y con el que comparar tus ideas.

Por eso los ejercicios son **de análisis**: importa tanto lo que construyes como **cómo has llegado hasta ahí**. Lo que vas a entregar es un informe sencillo, con esquemas, que cuente ese proceso.

1.1. Widgets que puedes usar

Usa solo los widgets de esta tabla. Son los que hemos visto en clase y bastan para resolver todos los ejercicios.

Para…	Widgets permitidos
Estructura de la app	`MaterialApp`, `Scaffold`, `AppBar`
Colocar elementos	`Center`, `Column`, `Row`, `Padding`, `SizedBox`, `Container`
Mostrar contenido	`Text`, `Icon`
Botones	`ElevatedButton`, `OutlinedButton`, `TextButton`, `IconButton`
Tus propios widgets	`StatelessWidget`, `StatefulWidget` (con `State` y `setState()`)
Comunicar widgets	Parámetros del constructor y funciones de tipo `VoidCallback`

> **SI LA IA USA OTROS WIDGETS**
>
> Las IA suelen proponer widgets más avanzados (`ListView`, `Card`, `GestureDetector`, `Provider`…). Si aparece alguno que no está en la tabla, tienes dos opciones: **sustituirlo** por uno permitido o **justificar** en el informe por qué lo necesitas y explicar qué hace. Lo que no vale es dejarlo sin entenderlo.

1.2. Cómo trabajar con la IA

Todos los ejercicios siguen el mismo método de cinco pasos:



Figura 1. *El método de trabajo de todos los ejercicios.*

1. **Analiza tú primero.** Antes de abrir la IA, escribe en el informe lo que piensas: tu árbol, tu predicción, tu clasificación. No pasa nada si te equivocas: es el punto de partida.
2. **Pregunta a la IA.** Copia en el informe los *prompts* (preguntas) importantes, tal y como los escribiste, y resume lo que contestó.
3. **Comprueba.** Ejecuta el código y mira si hace lo que dice la IA. Una IA puede sonar muy segura y estar equivocada.
4. **Corrige.** Si algo no encaja, vuelve a preguntar o arréglalo tú. Anota qué cambiaste.

5. **Explica con esquemas.** Dibuja árboles de widgets y flujos de datos que demuestren que entiendes el resultado.

1.3. Preguntar bien a la IA

La calidad de la respuesta depende mucho de la pregunta. Compara:

Pregunta poco útil	Pregunta útil
«Hazme el ejercicio 1.»	«Este es mi árbol de widgets para una pantalla de perfil. ¿Qué widget sobra o falta y por qué?»
«Arregla este código.»	«Explícame por qué este texto no cambia al pulsar el botón. No me des el código corregido.»
«¿Qué es setState?»	«¿Qué pasa en Flutter, paso a paso, desde que pulso un botón que llama a setState hasta que cambia la pantalla?»
«Hazlo mejor.»	«Divide este build() en widgets más pequeños usando solo StatelessWidget y explícame cada uno.»

NORMAS DE USO

Indica en el informe qué herramienta de IA has usado.

No copies nada que no sepas explicar: en la corrección se te puede pedir que expliques cualquier línea.

Si la IA se equivoca, ¡apúntalo! Detectar y explicar un error de la IA es de lo que más se valora.

1.4. El informe

Entregarás **un único documento PDF** con un apartado por ejercicio. Cada apartado tiene siempre estas cinco partes:

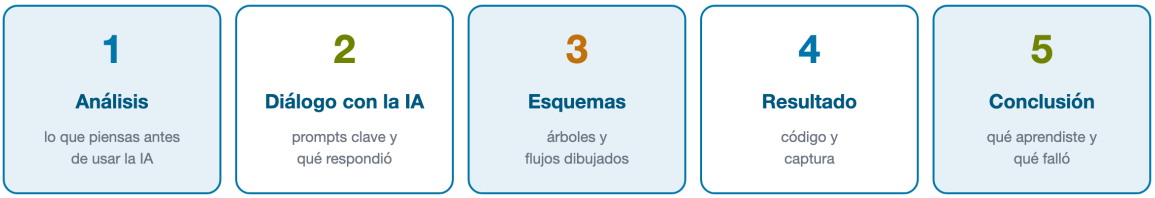


Figura 2. *Estructura de cada apartado del informe.*

- **Extensión orientativa:** una o dos páginas por ejercicio. Mejor claro que largo.
- **Esquemas:** pueden estar hechos a mano (foto nítida) o con una herramienta como draw.io, Excalidraw o Mermaid.
- **Código:** incluye solo las partes importantes en el informe. El proyecto completo se entrega aparte (carpeta comprimida o enlace al repositorio).
- **Capturas:** de la aplicación funcionando, y de la conversación con la IA solo si aportan algo.

1.5. Cómo se evalúa

Criterio	Peso	Qué se valora
Análisis propio	25 %	Se ve lo que pensaste **antes** de usar la IA, con tus propias palabras.
Uso crítico de la IA	25 %	Preguntas claras, respuestas comprobadas y errores de la IA detectados y explicados.
Esquemas	20 %	Árboles y flujos correctos, legibles y coherentes con el código.
Funcionamiento	20 %	El código se ejecuta, cumple los requisitos y usa solo los widgets permitidos.
Claridad	10 %	El informe está ordenado, es fácil de leer y no tiene relleno.

Ejercicio 1. El detective de árboles

Objetivo	Tiempo orientativo	Uso de la IA
Leer una pantalla y deducir el árbol de widgets que la construye.	1 sesión	Para comparar tu árbol con el suyo.

Esta es la pantalla de perfil de una aplicación. No tienes su código: solo la imagen. Tu trabajo es averiguar **qué widgets la forman y cómo están anidados**, y después construirla.

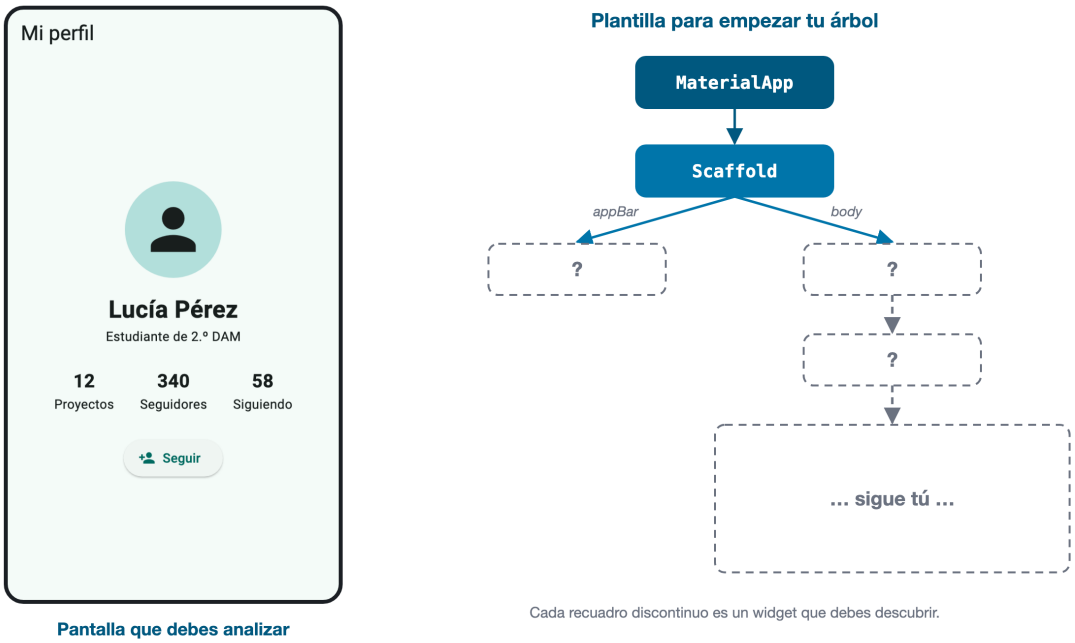


Figura 3. *La pantalla que debes analizar y una plantilla para empezar el árbol.*

Tareas

1. **Sin IA:** dibuja el árbol de widgets completo que crees que genera esta pantalla, usando solo widgets permitidos. Distingue con dos colores los widgets **de layout** (los que colocan) y los **visibles** (los que se ven).
2. Hay una parte de la pantalla que se **repite tres veces** con datos distintos. Rodéala en tu árbol y explica qué cambia entre una repetición y otra.
3. **Con IA:** describe la pantalla a la IA (o pásale la imagen, si tu herramienta lo permite) y pídele su propio árbol de widgets. No le enseñes el tuyo todavía.
4. Compara los dos árboles en una tabla: qué widgets coinciden, cuáles son distintos y cuál de las dos versiones te parece más sencilla y por qué.
5. Programa la pantalla. La parte que se repite debe ser **tu propio** `StatelessWidget`, que reciba sus datos por el constructor.
6. Activa `debugPaintSizeEnabled = true;` (importando `package:flutter/rendering.dart`), haz una captura y comprueba si los recuadros coinciden con tu árbol final.

Preguntas para analizar

- ¿Qué ventaja tiene crear un widget propio para la parte que se repite, en lugar de escribirla tres veces?
- ¿Has tenido que cambiar tu árbol inicial después de programar? ¿Qué te había pasado desapercibido?
- Si la IA propuso algún widget que no está permitido, ¿por cuál lo cambiaste?

QUÉ DEBE APARECER EN TU INFORME

☐ Tu árbol inicial (antes de la IA).

☐ El árbol de la IA y la tabla comparativa.

☐ El código de tu widget para la parte repetida.

☐ Captura normal y captura con debugPaintSizeEnabled.

☐ Respuestas a las preguntas y conclusión.

Ejercicio 2. ¿Con estado o sin estado?

Objetivo	Tiempo orientativo	Uso de la IA
Decidir con criterio si un widget debe ser StatelessWidget o StatefulWidget.	1 sesión	Para contrastar tu clasificación y discutirla.

Elegir bien el tipo de widget es una de las primeras decisiones al programar una pantalla. Para decidir, hazte siempre esta pregunta:

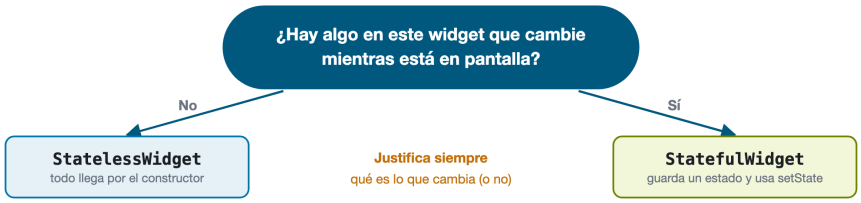


Figura 4. *La pregunta clave para elegir el tipo de widget.*

Copia esta tabla en tu informe y complétala. Cada fila describe un widget que podría aparecer en una aplicación:

N.º	Descripción del widget	¿Stateless o Stateful?	¿Qué cambia? (o por qué nada cambia)
1	Un texto con el nombre de la aplicación en la barra superior.		
2	Un interruptor para activar el modo oscuro de una pantalla.		
3	Una tarjeta que muestra el nombre de un contacto que recibe de su padre.		
4	Un contador de vasos de agua que suma uno al pulsar un botón.		
5	Una estrella de «favorito» que se rellena al tocarla.		
6	Un botón que recibe por parámetro la función que debe ejecutar al pulsarlo.		
7	Un reloj que muestra la hora y se actualiza cada segundo.		
8	Un campo de texto que muestra debajo lo que vas escribiendo.		

Tareas

1. **Sin IA:** completa la tabla. En la última columna justifica tu decisión en una frase.
2. **Con IA:** pide a la IA que clasifique los mismos ocho widgets y que justifique cada respuesta. No le enseñes tus respuestas.
3. Haz una tabla con los casos en los que **no estáis de acuerdo**. Para cada uno, decide quién tiene razón y explica por qué. Si sigues con dudas, programa un ejemplo pequeño para comprobarlo.
4. Elige **un** widget de la lista que sea sin estado y **uno** que sea con estado, y prográmalos. Muestra los dos funcionando en la misma pantalla.
5. Para el widget con estado, dibuja un esquema que indique qué variable forma su estado y en qué momento se llama a `setState()`.

Preguntas para analizar

- Un widget puede mostrar datos que cambian con el tiempo y, aun así, ser `StatelessWidget`. ¿Cómo es posible? Busca un ejemplo en la tabla.
- ¿Qué inconveniente tendría hacer todos los widgets `StatefulWidget` «por si acaso»?

QUÉ DEBE APARECER EN TU INFORME

☐ Tu tabla inicial.

☐ La clasificación de la IA y la tabla de desacuerdos razonada.

☐ Código y captura de los dos widgets programados.

☐ Esquema del estado del widget con estado.

☐ Respuestas a las preguntas y conclusión.

Ejercicio 3. El contador que no cuenta

Objetivo	Tiempo orientativo	Uso de la IA
Entender cómo se actualiza la pantalla y detectar errores con setState().	1 sesión	Para que explique el error, sin que lo arregle por ti.

Un compañero ha programado un contador muy sencillo, pero dice que «no funciona». Este es su código completo:

lib/main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(
    const MaterialApp(
      home: Scaffold(body: Center(child: Contador())),
    ),
  );
}

class Contador extends StatefulWidget {
  const Contador({super.key});

  @override
  State<Contador> createState() => _ContadorState();
}

class _ContadorState extends State<Contador> {
  int pulsaciones = 0;

  @override
  Widget build(BuildContext context) {
    int total = 0;

    return Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Text('Pulsaciones: $pulsaciones'),
        Text('Total: $total'),
        ElevatedButton(
          onPressed: () {
            pulsaciones++;
            total++;
          },
          child: const Text('Sumar'),
        ),
      ],
    );
  }
}
```

> **ATENCIÓN**
>
> Cuidado: puede haber **más de un problema**. No te quedes con el primero que encuentres.

Tareas

1. **Predicción, sin ejecutar y sin IA:** escribe qué crees que mostrará la pantalla al abrir la aplicación y después de pulsar «Sumar» tres veces. Explica por qué lo crees.
2. **Ejecuta** el código y anota lo que ocurre de verdad. ¿Acertaste?
3. Copia en tu informe el ciclo de la figura y **complétalo**: qué falta en cada hueco y en qué punto se rompe el ciclo en este código.

4. **Con IA:** pide a la IA que te explique qué está pasando, **pidiéndole expresamente que no te dé el código corregido**. Valora su explicación: ¿coincide con la tuya?, ¿ha encontrado todos los problemas?, ¿hay algo que no te haya convencido?
5. Corrige tú el código. Demuestra que funciona con una captura después de pulsar tres veces.
6. Explica cada cambio que has hecho en una tabla: línea original, línea nueva y motivo.

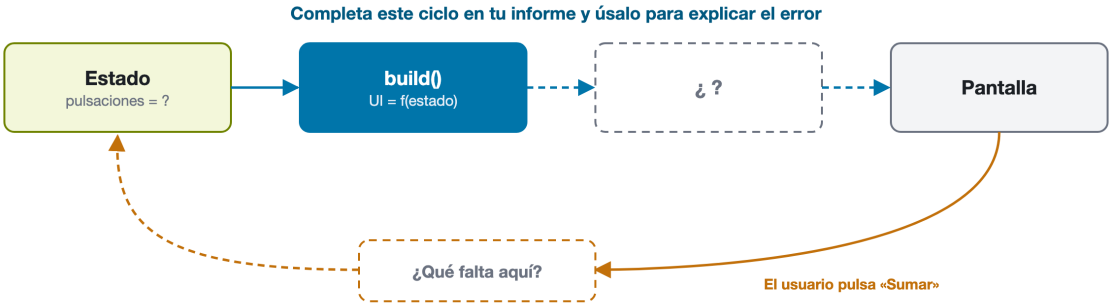


Figura 5. *Ciclo de actualización de la pantalla, con huecos para completar.*

Preguntas para analizar

- ¿Por qué Flutter no actualiza la pantalla automáticamente cuando cambia el valor de una variable?
- ¿Qué diferencia hay entre declarar una variable dentro del método `build()` y declararla como atributo de la clase `State`? Relaciónalo con lo que has visto.

QUÉ DEBE APARECER EN TU INFORME

☐ Tu predicción y lo que ocurrió al ejecutar.

☐ El ciclo completado.

☐ La explicación de la IA y tu valoración.

☐ El código corregido, la tabla de cambios y la captura.

☐ Respuestas a las preguntas y conclusión.

Ejercicio 4. Los datos bajan, los eventos suben

Objetivo	Tiempo orientativo	Uso de la IA
Diseñar dónde vive el estado y cómo se comunican los widgets.	1–2 sesiones	Para programar a partir de tu diseño y revisar su código.

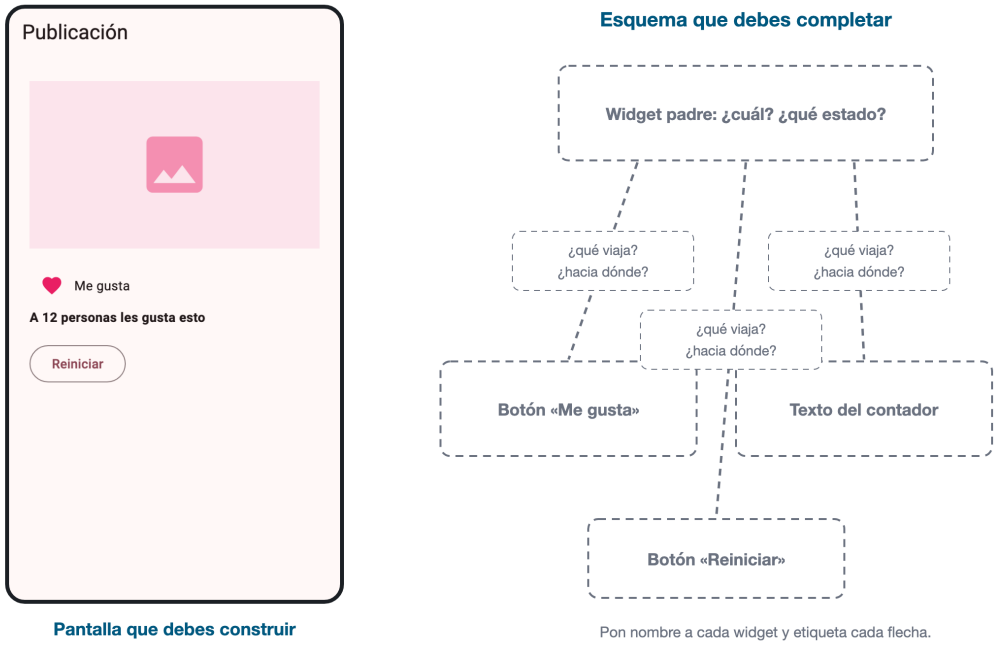


Figura 6. *La pantalla que debes construir y el esquema de diseño que tienes que completar.*

Qué debe hacer la pantalla

- Al pulsar el corazón, se marca (relleno) o se desmarca (vacío), y el número de «me gusta» sube o baja en uno.
- El texto «A N personas les gusta esto» muestra siempre el número correcto.
- El botón «Reiniciar» pone el contador a cero y deja el corazón desmarcado.

Restricciones

- En toda la pantalla solo puede haber **un** `StatefulWidget`.
- El botón del corazón, el texto del contador y el botón «Reiniciar» deben ser **tus propios** `StatelessWidget`.
- Ningún widget hijo puede llamar a `setState()` ni modificar el estado directamente.

Tareas

1. **Diseño, sin IA:** copia y completa el esquema de la figura. Indica qué widget guarda el estado, qué variables tiene y, en cada línea, qué viaja (un dato o un aviso) y en qué sentido.
2. **Con IA:** pide a la IA el código **dándole tu diseño y las restricciones**. Copia tu *prompt* en el informe.
3. Revisa el código de la IA con la lista de comprobación de abajo. Para cada punto, indica «sí» o «no» y la línea de código que lo demuestra.
4. Si algo no se cumple, pide la corrección (o corrígelo tú) y anota qué ha cambiado.
5. Dibuja el esquema final, ya con los nombres reales de tus widgets, y compáralo con tu diseño inicial.

Lista de comprobación	¿Se cumple?	Línea que lo demuestra
Hay un único StatefulWidget en toda la pantalla.		
Los tres widgets hijos son StatelessWidget propios.		
Los atributos de los widgets hijos son final.		
Los hijos avisan al padre mediante funciones recibidas por el constructor.		
setState() solo aparece en el widget padre.		
Solo se usan widgets permitidos.		

Preguntas para analizar

- ¿Qué problema aparecería si el botón del corazón y el texto tuvieran cada uno su propio estado?
- ¿Qué ventaja tiene que los widgets hijos no sepan «qué pasa» cuando se les pulsa?

QUÉ DEBE APARECER EN TU INFORME

☐ Tu esquema de diseño inicial.

☐ El prompt usado y la lista de comprobación completada.

☐ Correcciones realizadas (si las hubo).

☐ Esquema final, código principal y capturas de la app funcionando.

☐ Respuestas a las preguntas y conclusión.

Ejercicio 5. Revisor de código de la IA

Objetivo	Tiempo orientativo	Uso de la IA
Evaluar con criterio el código que genera una IA y mejorarlo paso a paso.	1–2 sesiones	La IA escribe la primera versión; tú la auditas y diriges las mejoras.

Esta vez la IA hará la primera versión del código, y tú actuarás como **revisor**: comprobarás si cumple lo que se pide, si usa solo lo permitido y si lo entiendes de verdad. Vas a construir una tarjeta de presentación personal como la del boceto:

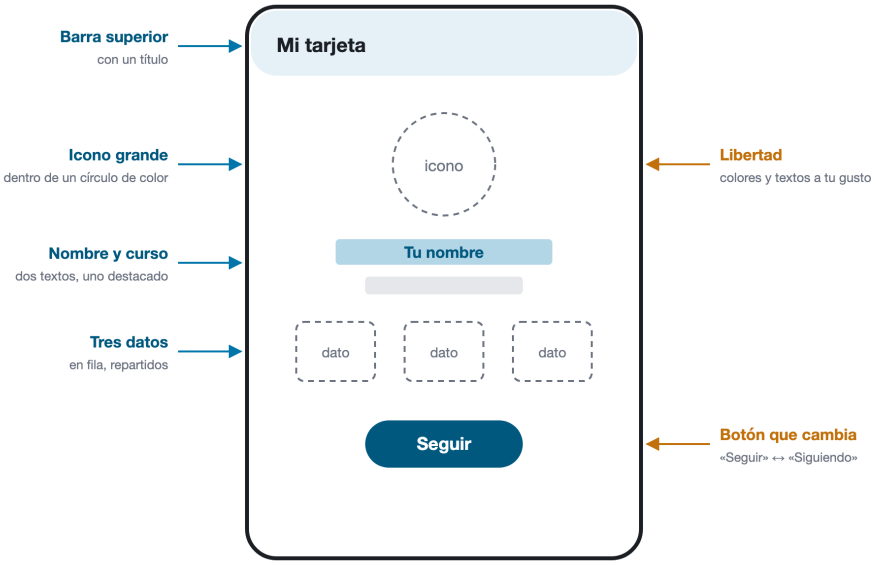


Figura 7. *Boceto de la tarjeta de presentación. Los textos y colores los decides tú.*

Requisitos

1. Una barra superior con un título.
2. Un icono grande dentro de un círculo de color.
3. Tu nombre (destacado) y tu ciclo formativo debajo.
4. Una fila con tres datos (por ejemplo, proyectos, horas de prácticas y lenguajes) repartidos a lo ancho.
5. Un botón que alterna entre «Seguir» y «Siguiendo» cada vez que se pulsa.
6. Solo widgets permitidos y un único `StatefulWidget`.

Tareas

1. **Escribe tú el prompt** con todos los requisitos. Cópialo en el informe y explica por qué lo has redactado así.
2. **Audita la primera versión** de la IA con una tabla: requisito, ¿se cumple?, evidencia (línea de código o captura).
3. Señala cualquier widget no permitido y cualquier línea que **no entiendas**. Pregunta a la IA por ellas y explica con tus palabras lo que has aprendido.
4. **Mejora la versión en al menos dos rondas.** En cada ronda anota: qué pediste, qué cambió y si el cambio fue a mejor.
5. Dibuja el **árbol de widgets final** y marca con un color la parte que tiene estado.

Preguntas para analizar

- Di tres cosas que la IA hizo bien a la primera y tres que tuviste que corregir o pedir de nuevo.
- Calcula de forma aproximada qué parte del código final entiendes por completo (en porcentaje) y explica qué te falta para llegar al 100 %.
- ¿Qué cambiarías de tu primer prompt si tuvieras que repetir el ejercicio?

QUÉ DEBE APARECER EN TU INFORME

☐ Tu prompt inicial y su justificación.

☐ La tabla de auditoría de la primera versión.

☐ El registro de las rondas de mejora.

☐ El árbol de widgets final y capturas de la tarjeta (con «Seguir» y con «Siguiendo»).

☐ Respuestas a las preguntas y conclusión.

Antes de entregar

Repasa esta lista antes de subir tu trabajo:

Comprobación	✓
El informe es un único PDF, con un apartado por ejercicio y tu nombre en la portada.	
Cada apartado tiene las cinco partes: análisis, diálogo con la IA, esquemas, resultado y conclusión.	
Los esquemas se leen bien (si son fotos, están enfocadas y derechas).	
Has indicado qué herramienta de IA has usado.	
Todo el código se ejecuta sin errores y usa solo widgets permitidos (o está justificado).	
Sabrías explicar en voz alta cualquier línea del código que entregas.	
Has entregado también el proyecto completo (carpeta comprimida o enlace al repositorio).	